

Chapter 12

Sorting Algorithm

ผศ. ดร. สีดดา อินทรโสธรนันท์

สาขาวิชาวิทยาการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

เทคนิคการจัดเรียงข้อมูล

1. การเรียงลำดับแบบเลือก (Selection Sort)
2. การเรียงลำดับแบบฟองอากาศ (Bubble Sort)
3. การเรียงลำดับแบบแทรก (Insertion Sort)
4. การเรียงลำดับแบบเชลล์ (Shell Sort)
5. การเรียงลำดับแบบผสาน (Merge Sort)
6. การเรียงลำดับแบบเร็ว (Quick Sort)
7. การเรียงลำดับแบบฮีป (Heap Sort)



1. การเรียงลำดับแบบเลือก (Selection Sort)

หลักการทำงาน

- Selection Sort เป็นอัลกอริทึมการเรียงลำดับที่เรียบง่าย
- หลักการทำงานคือ "ค้นหาข้อมูลที่มีค่าน้อยที่สุด (หรือมากที่สุด) ในส่วนที่ยังไม่ได้เรียงลำดับ แล้วนำมาสลับตำแหน่งกับข้อมูลตัวแรกของส่วนนั้น" ทำซ้ำแบบนี้ไปเรื่อย ๆ จนกว่าข้อมูลจะเรียงลำดับครบทั้งหมด
- ข้อดีคือ ความเรียบง่าย เหมาะกับข้อมูลที่มีการเปลี่ยนแปลงน้อย
- แต่มีข้อจำกัดด้านประสิทธิภาพสำหรับข้อมูลขนาดใหญ่
- Time Complexity $O(n^2)$ เพราะต้องวนลูปค้นหาค่าน้อยที่สุดซ้ำๆ

ขั้นตอนการทำงาน

- การเรียงลำดับจาก น้อยไปมาก มีขั้นตอนดังนี้:
 1. แบ่งข้อมูลออกเป็น 2 ส่วน: ส่วนที่เรียงลำดับแล้ว (เริ่มต้นจะมีสมาชิกเป็น 0 ตัว) และส่วนที่ยังไม่ได้เรียงลำดับ (เริ่มต้นคือข้อมูลทั้งหมด)
 2. ค้นหาอย่างน้อยที่สุด: วนหาข้อมูลที่มีค่าน้อยที่สุดใน "ส่วนที่ยังไม่ได้เรียงลำดับ"
 3. สลับตำแหน่ง: นำค่าน้อยที่สุดที่หาได้ สลับตำแหน่งกับข้อมูลตัวแรกของส่วนที่ยังไม่ได้เรียงลำดับ
 4. ขยายส่วนที่เรียงแล้ว: ขยับเส้นแบ่งให้ส่วนที่เรียงลำดับแล้วเพิ่มขึ้น 1 ตัว
 5. ทำซ้ำ: ทำตามข้อ 2–4 จนกว่าส่วนที่ยังไม่ได้เรียงลำดับจะเหลือเพียงข้อมูลตัวเดียว

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

รอบที่ 1:

64	25	12	22	11
----	----	----	----	----

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 11

สลับ 11 กับ 64 (ตัวแรก) ผลลัพธ์:

11	25	12	22	64
----	----	----	----	----

ตัวอย่างการทำงาน

รอบที่ 2:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 12

สลับ 12 กับ 25 ผลลัพธ์:

11	12	25	22	64
----	----	----	----	----

รอบที่ 3:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 22

สลับ 22 กับ 25 ผลลัพธ์:

11	12	22	25	64
----	----	----	----	----

ตัวอย่างการทำงาน

รอบที่ 4:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 25

อยู่ตำแหน่งเดิมไม่ต้องสลับ

ผลลัพธ์จัดเรียงข้อมูลเรียบร้อยแล้ว:

11	12	22	25	64
----	----	----	----	----

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 1:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 9

สลับ 9 กับ 25 (ตัวแรก) ผลลัพธ์:

9	32	12	55	25	18	50	45
---	----	----	----	----	----	----	----

ตัวอย่างการทำงาน

รอบที่ 2:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 12

สลับ 12 กับ 32 ผลลัพธ์:

9	12	32	55	25	18	50	45
---	----	----	----	----	----	----	----

รอบที่ 3:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 18

สลับ 18 กับ 32 ผลลัพธ์:

9	12	18	55	25	32	50	45
---	----	----	----	----	----	----	----

ตัวอย่างการทำงาน

รอบที่ 4:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 25

สลับ 25 กับ 55 ผลลัพธ์:

9	12	18	25	55	32	50	45
---	----	----	----	----	----	----	----

รอบที่ 5:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 32

สลับ 32 กับ 55 ผลลัพธ์:

9	12	18	25	32	55	50	45
---	----	----	----	----	----	----	----

ตัวอย่างการทำงาน

รอบที่ 6:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 45

สลับ 45 กับ 55 ผลลัพธ์:

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

รอบที่ 7:

ค้นหาค่าน้อยที่สุดได้ผลลัพธ์เป็น 50

อยู่ตำแหน่งเดิมไม่ต้องสลับ ผลลัพธ์จัดเรียงข้อมูลเรียบร้อยแล้ว:

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----



2. การเรียงลำดับแบบฟองอากาศ (Bubble Sort)

หลักการทำงาน

- Selection Sort เป็นอัลกอริทึมการเรียงลำดับที่เรียบง่าย
- หลักการทำงานคือ "เปรียบเทียบข้อมูลคู่ที่อยู่ติดกันทีละคู่ แล้วสลับตำแหน่งถ้าค่านั้นเรียงไม่ถูกต้อง" ขบวนการนี้จะทำซ้ำไปเรื่อยๆ จนกว่าข้อมูลตัวที่มากที่สุด (หรือน้อยที่สุด) จะค่อยๆ ย้ายไปอยู่อีกฝั่งหนึ่ง เหมือนกับฟองอากาศที่ลอยขึ้นสู่ผิวน้ำ
- เปรียบเทียบกับ Selection Sort จะพบว่า Bubble Sort จะมีการ สลับข้อมูล (Swaps) บ่อยกว่ามากในระหว่างวนลูป แต่สามารถจบการทำงานได้เร็วกว่าถ้าข้อมูลเรียงมาอยู่แล้ว
- Time Complexity $O(n^2)$

ขั้นตอนการทำงาน

- การเรียงลำดับจาก น้อยไปมาก มีขั้นตอนดังนี้:

1. เริ่มจากคู่แรก: เปรียบเทียบข้อมูลตำแหน่งที่ 1 กับตำแหน่งที่ 2
2. สลับถ้าไม่ถูกต้อง: ถ้าข้อมูลตัวซ้าย มากกว่า ข้อมูลตัวขวา ให้สลับตำแหน่งกัน
3. เลื่อนไปคู่ถัดไป: เลื่อนไปเปรียบเทียบตำแหน่งที่ 2 กับ 3, 3 กับ 4 ไปเรื่อยๆ จนสุดแถว
4. เมื่อเปรียบเทียบจนสุดแถวในรอบแรก ข้อมูลที่มีค่ามากที่สุดจะถูกดันไปอยู่ที่ตำแหน่งสุดท้าย
5. ทำซ้ำรอบใหม่: เริ่มเปรียบเทียบตั้งแต่คู่แรกใหม่ โดยลดขอบเขตการเปรียบเทียบลงทีละ 1 (เพราะตัวท้ายๆ เรียงเสร็จแล้ว) ทำจนกระทั่งไม่มีการสลับตำแหน่งเกิดขึ้นอีกเลยในรอบนั้น

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

64	25	12	22	11
----	----	----	----	----

รอบที่ 1:

64	25	12	22	11
----	----	----	----	----

เปรียบเทียบ 64 กับ 25 ($64 > 25$) สลับ

25	64	12	22	11
----	----	----	----	----

25	64	12	22	11
----	----	----	----	----

เปรียบเทียบ 64 กับ 12 ($64 > 12$) สลับ

25	12	64	22	11
----	----	----	----	----

25	12	64	22	11
----	----	----	----	----

เปรียบเทียบ 64 กับ 22 ($64 > 22$) สลับ

25	12	22	64	11
----	----	----	----	----

25	12	22	64	11
----	----	----	----	----

เปรียบเทียบ 64 กับ 11 ($64 > 11$) สลับ

25	12	22	11	64
----	----	----	----	----

จบการทำงานรอบที่ 1

ตัวอย่างการทำงาน

รอบที่ 2:

25	12	22	11	64
12	25	22	11	64
12	22	25	11	64

เปรียบเทียบ 25 กับ 12 ($25 > 12$) สลับ

เปรียบเทียบ 25 กับ 22 ($25 > 22$) สลับ

เปรียบเทียบ 25 กับ 11 ($25 > 11$) สลับ

12	25	22	11	64
12	22	25	11	64
12	22	11	25	64

จบการทำงานรอบที่ 2

ตัวอย่างการทำงาน

รอบที่ 3:

12	22	11	25	64
----	----	----	----	----

เปรียบเทียบ 12 กับ 22 (เรียงถูกแล้ว) ไม่สลับ

12	22	11	25	64
----	----	----	----	----

เปรียบเทียบ 22 กับ 11 ($22 > 11$) สลับ

12	11	22	25	64
----	----	----	----	----

จบการทำงานรอบที่ 3

รอบที่ 4:

12	11	22	25	64
----	----	----	----	----

เปรียบเทียบ 12 กับ 11 ($12 > 11$) สลับ

11	12	22	25	64
----	----	----	----	----

จบการทำงานรอบที่ 4 ข้อมูลจัดเรียงเรียบร้อยแล้ว

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 1:

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

เปรียบเทียบ 25 กับ 32 (เรียงถูกแล้ว) ไม่สลับ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

เปรียบเทียบ 32 กับ 12 ($32 > 12$) สลับ

25	12	32	55	9	18	50	45
----	----	----	----	---	----	----	----

25	12	32	55	9	18	50	45
----	----	----	----	---	----	----	----

เปรียบเทียบ 32 กับ 55 (เรียงถูกแล้ว) ไม่สลับ

25	12	32	55	9	18	50	45
----	----	----	----	---	----	----	----

เปรียบเทียบ 55 กับ 9 ($55 > 9$) สลับ

25	12	32	9	55	18	50	45
----	----	----	---	----	----	----	----

25	12	32	9	55	18	50	45
----	----	----	---	----	----	----	----

เปรียบเทียบ 55 กับ 18 ($55 > 18$) สลับ

25	12	32	9	18	55	50	45
----	----	----	---	----	----	----	----

25	12	32	9	18	55	50	45
----	----	----	---	----	----	----	----

เปรียบเทียบ 55 กับ 50 ($55 > 50$) สลับ

25	12	32	9	18	50	55	45
----	----	----	---	----	----	----	----

25	12	32	9	18	50	55	45
----	----	----	---	----	----	----	----

เปรียบเทียบ 55 กับ 45 ($55 > 45$) สลับ

25	12	32	9	18	50	45	55
----	----	----	---	----	----	----	----

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 2:

25	12	32	9	18	50	45	55
12	25	32	9	18	50	45	55
12	25	32	9	18	50	45	55
12	25	9	32	18	50	45	55
12	25	9	18	32	50	45	55
12	25	9	18	32	50	45	55

เปรียบเทียบ 25 กับ 12 ($25 > 12$) สลับ

12	25	32	9	18	50	45	55
----	----	----	---	----	----	----	----

เปรียบเทียบ 25 กับ 32 (เรียงถูกต้องแล้ว) ไม่สลับ

เปรียบเทียบ 32 กับ 9 ($32 > 9$) สลับ

12	25	9	32	18	50	45	55
----	----	---	----	----	----	----	----

เปรียบเทียบ 32 กับ 18 ($32 > 18$) สลับ

12	25	9	18	32	50	45	55
----	----	---	----	----	----	----	----

เปรียบเทียบ 32 กับ 50 (เรียงถูกต้องแล้ว) ไม่สลับ

เปรียบเทียบ 50 กับ 45 ($50 > 45$) สลับ

12	25	9	18	32	45	50	55
----	----	---	----	----	----	----	----

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 3:

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

12	25	9	18	32	45	50	55
----	----	---	----	----	----	----	----

เปรียบเทียบ 12 กับ 25 (เรียงถูกต้องแล้ว) ไม่สลับ

12	25	9	18	32	45	50	55
----	----	---	----	----	----	----	----

เปรียบเทียบ 25 กับ 9 ($25 > 9$) สลับ

12	9	25	18	32	45	50	55
----	---	----	----	----	----	----	----

12	9	25	18	32	45	50	55
----	---	----	----	----	----	----	----

เปรียบเทียบ 25 กับ 18 ($25 > 18$) สลับ

12	9	18	25	32	45	50	55
----	---	----	----	----	----	----	----

12	9	18	25	32	45	50	55
----	---	----	----	----	----	----	----

เปรียบเทียบ 25 กับ 32 (เรียงถูกต้องแล้ว) ไม่สลับ

12	9	18	25	32	45	50	55
----	---	----	----	----	----	----	----

เปรียบเทียบ 32 กับ 45 (เรียงถูกต้องแล้ว) ไม่สลับ

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 4:

12	9	18	25	32	45	50	55
----	---	----	----	----	----	----	----

เปรียบเทียบ 12 กับ 9 ($12 > 9$) สลับ

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

เปรียบเทียบ 12 กับ 18 (เรียงถูกต้องแล้ว) ไม่สลับ

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

เปรียบเทียบ 18 กับ 25 (เรียงถูกต้องแล้ว) ไม่สลับ

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

เปรียบเทียบ 25 กับ 32 (เรียงถูกต้องแล้ว) ไม่สลับ

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 5:

9	12	18	25	32	45	50	55
9	12	18	25	32	45	50	55
9	12	18	25	32	45	50	55

เปรียบเทียบ 9 กับ 12 (เรียงถูกแล้ว) ไม่สลับ

เปรียบเทียบ 12 กับ 18 (เรียงถูกแล้ว) ไม่สลับ

เปรียบเทียบ 18 กับ 25 (เรียงถูกแล้ว) ไม่สลับ

เมื่อไม่มีการสลับตำแหน่งเลยทั้งรอบ จะหยุดการทำงาน



3. การเรียงลำดับแบบแทรก (Insertion Sort)

หลักการทำงาน

- Insertion Sort มีหลักการทำงานเหมือนกับ "การจัดเรียงไฟในมือ" โดยหยิบไฟทีละใบจากกอง แล้วนำไป "แทรก" ในตำแหน่งที่ถูกต้องของไฟที่เรียงไว้แล้วในมือ
- Time Complexity $O(n^2)$
- จุดเด่น มีประสิทธิภาพสูงมากกับชุดข้อมูลขนาดเล็ก หรือชุดข้อมูลที่ เกือบเรียงลำดับสมบูรณ์อยู่แล้ว

ขั้นตอนการทำงาน (เรียงจากน้อยไปมาก)

1. แบ่งข้อมูลเป็น 2 ส่วน: ส่วนที่เรียงลำดับแล้ว (เริ่มต้นมีข้อมูลตัวแรกสุดเพียง 1 ตัว) ส่วนที่ยังไม่ได้เรียงลำดับ (ข้อมูลที่เหลือตั้งแต่ตัวที่ 2 เป็นต้นไป)
2. หยิบข้อมูลมา 1 ตัว: หยิบข้อมูลตัวแรกสุดของส่วนที่ยังไม่ได้เรียงลำดับออกมา (เรียกว่า Key)
3. เปรียบเทียบและถอยหลัง: นำ Key ไปเปรียบเทียบกับข้อมูลในส่วนที่เรียงลำดับแล้ว โดยไล่เปรียบเทียบจากขวาไปซ้าย (จากค่ามากไปน้อย)
4. ขยับช่องว่าง: ถ้าข้อมูลในส่วนที่เรียงแล้วมีค่า มากกว่า Key ให้ขยับข้อมูลตัวนั้นไปทางขวา 1 ช่อง เพื่อสร้างช่องว่าง
5. แทรกข้อมูล (Insert): เมื่อเจอข้อมูลที่มีค่าน้อยกว่าหรือเท่ากับ Key (หรือถอยไปจนสุดทางซ้าย) ให้วาง Key ลงในช่องว่างนั้นทันที
6. ทำซ้ำ: ทำตามข้อ 2–5 จนกว่าข้อมูลทุกตัวจะถูกแทรกเข้าสู่ส่วนที่เรียงลำดับแล้วจนหมด

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
เริ่มต้น ส่วนที่เรียงแล้วคือ **25** ส่วนที่ยังไม่เรียง

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

32	12	55	9	18	50	45
----	----	----	---	----	----	----

รอบที่ 1 หยิบ 32 มา

$32 > 25$ ดังนั้น ไม่ต้องขยับ แต่ให้นำ 32 มาต่อ 25

25	32
----	----

รอบที่ 2 หยิบ 12 มา

$12 < 32$ ต้องขยับ 32 ไป 1 ตำแหน่ง

25		32
----	--	----

$12 < 25$ ต้องขยับ 25 ไป 1 ตำแหน่ง แล้วจึงนำ 12 ไปใส่ลง

12	25	32
----	----	----

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 3 หยิบ 55 มา

$55 > 32$ ดังนั้น ไม่ต้องขยับ แต่ให้นำ 55 มาต่อ 32

12	25	32	55
----	----	----	----

รอบที่ 4 หยิบ 9 มา

$9 < 55$ ต้องขยับ 55 ไป 1 ตำแหน่ง

$9 < 32$ ต้องขยับ 32 ไป 1 ตำแหน่ง

$9 < 25$ ต้องขยับ 25 ไป 1 ตำแหน่ง

$9 < 12$ ต้องขยับ 12 ไป 1 ตำแหน่ง แล้วจึงนำ 9 ใ้ลงไป

12	25	32		55
----	----	----	--	----

12	25		32	55
----	----	--	----	----

12		25	32	55
----	--	----	----	----

9	12	25	32	55
---	----	----	----	----

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 5 หยิบ 18 มา

$18 < 55$ ต้องขยับ 55 ไป 1 ตำแหน่ง

$18 < 32$ ต้องขยับ 32 ไป 1 ตำแหน่ง

$18 < 25$ ต้องขยับ 25 ไป 1 ตำแหน่ง

$18 > 12$ ไม่ต้องขยับ 12 ให้นำ 18 ไปลงได้เลย

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

9	12	25	32		55
---	----	----	----	--	----

9	12	25		32	55
---	----	----	--	----	----

9	12		25	32	55
---	----	--	----	----	----

9	12	18	25	32	55
---	----	----	----	----	----

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 6 หยิบ 50 มา

$50 < 55$ ต้องขยับ 55 ไป 1 ตำแหน่ง

$50 > 32$ ไม่ต้องขยับ 32 ให้นำ 50 ไปลงได้เลย

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

9	12	18	25	32		55
---	----	----	----	----	--	----

9	12	18	25	32	50	55
---	----	----	----	----	----	----

- รอบที่ 7 หยิบ 45 มา

$45 < 55$ ต้องขยับ 55 ไป 1 ตำแหน่ง

$45 < 50$ ต้องขยับ 50 ไป 1 ตำแหน่ง

$45 > 32$ ไม่ต้องขยับ 32 ให้นำ 45 ไปลงได้เลย

9	12	18	25	32	50		55
---	----	----	----	----	----	--	----

9	12	18	25	32		50	55
---	----	----	----	----	--	----	----

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

4. การเรียงลำดับแบบเชลล์ (Shell Sort)

หลักการทำงาน

- Shell Sort ถูกคิดค้น โดย Donald Shell เป็นอัลกอริทึมที่นำ Insertion Sort มาปรับปรุงให้ทำงานได้เร็วขึ้น โดยแก้จุดอ่อนของ Insertion Sort ที่ต้องค่อยๆ ขยับข้อมูลที่ละ 1 ตำแหน่ง
- หลักการทำงานสำคัญคือ "ทำการเปรียบเทียบและสลับข้อมูลที่อยู่ห่างกันเป็นระยะทางหนึ่ง (เรียกว่า Gap) แล้วค่อยๆ ลดระยะห่างนั้นลงเรื่อย ๆ จนกระทั่ง Gap เท่ากับ 1" ซึ่งจะทำให้ข้อมูลโดยรวมถูกจัดให้อยู่ใกล้ตำแหน่งที่ถูกต้องอย่างรวดเร็วก่อนที่จะทำ Insertion Sort รอบสุดท้าย
- Time Complexity $O(n^2)$ หรือ $O(n^{1.5})$ ขึ้นอยู่กับลำดับชุดตัวเลข Gap ที่เลือกใช้
- จุดเด่น เร็วกว่า $O(n^2)$ ทั้งหมด (เช่น Bubble/Selection/Insertion) มาก โดยเฉพาะกับชุดข้อมูลขนาดปานกลาง และเขียนโค้ดได้ง่ายโดยไม่ต้องใช้ Memory เพิ่ม

ขั้นตอนการทำงาน

- การเรียงลำดับจาก น้อยไปมาก มีขั้นตอนดังนี้:
 1. กำหนดระยะห่าง (Gap): เริ่มต้นด้วยการตั้งค่า $Gap = \lfloor n/2 \rfloor$ โดย n คือจำนวนข้อมูลทั้งหมด
 2. จับคู่และเรียงลำดับข้ามช่อง: แบ่งข้อมูลตามระยะ Gap แล้วทำการเรียงลำดับข้อมูลในกลุ่มนั้นๆ (โดยใช้หลักการของ Insertion Sort)
 3. ลดค่า Gap: เมื่อจัดเรียงครบรอบแล้ว ให้ลดค่า Gap ลง เช่น หารสอง $\lfloor Gap/2 \rfloor$
 4. ทำซ้ำ: ทำตามข้อ 2 และ 3 ไปเรื่อยๆ
 5. รอบสุดท้าย (Gap=1) : ในรอบนี้จะเป็นการทำ Insertion Sort ตามปกติ แต่เนื่องจากข้อมูลส่วนใหญ่ถูกจัดเรียงมาเกือบสมบูรณ์ในรอบก่อนหน้านี้แล้ว ทำให้ทำงานเสร็จได้อย่างรวดเร็วมาก

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 1 $\text{Gap} = 8/2 = 4$

จับคู่ข้อมูลที่อยู่ห่างกัน 4 ช่อง

Index 0 กับ index 4 : 25 กับ 9 >>> สลับ 9 กับ 25

Index 1 กับ index 5 : 32 กับ 18 >>> สลับ 18 กับ 32

Index 2 กับ index 6 : 12 กับ 50 >>> เรียงถูกแล้ว

Index 3 กับ index 7 : 55 กับ 45 >>> สลับ 45 กับ 55

9	32	12	55	25	18	50	45
---	----	----	----	----	----	----	----

9	18	12	55	25	32	50	45
---	----	----	----	----	----	----	----

9	18	12	45	25	32	50	55
---	----	----	----	----	----	----	----

ข้อมูลที่จะเรียงรอบที่ 1

9	18	12	45	25	32	50	55
---	----	----	----	----	----	----	----

ตัวอย่างการทำงาน

- ข้อมูลที่จัดเรียงจากรอบที่ 1

9	18	12	45	25	32	50	55
---	----	----	----	----	----	----	----

รอบที่ 2 $\text{Gap} = 4/2 = 2$

จับคู่ข้อมูลที่อยู่ห่างกัน 2 ช่อง และเรียงลำดับที่ละกลุ่ม

กลุ่มที่ 1 : [index 0, index 2, index 4, index 6] : [9, 12, 25, 50]

กลุ่มที่ 2 : [index 1, index 3, index 5, index 7] : [18, 45, 32, 55]

กลุ่มที่ 1 เทียบ 9 กับ 12 >> ถูกแล้วไม่ต้องสลับ

เทียบ 12 กับ 25 >> ถูกแล้วไม่ต้องสลับ

เทียบ 25 กับ 50 >> ถูกแล้วไม่ต้องสลับ

9	18	12	45	25	32	50	55
---	----	----	----	----	----	----	----

ตัวอย่างการทำงาน

กลุ่มที่ 2 : [index 1, index 3, index 5, index 7] : [18, 45, 32, 55]

กลุ่มที่ 2 เทียบ 18 กับ 45 >> ถูกแล้วไม่ต้องสลับ

เทียบ 45 กับ 32 >> สลับ 32 กับ 45

9	18	12	32	25	45	50	55
---	----	----	----	----	----	----	----

แล้วนำ 32 เทียบกับ 18 ถูกแล้วไม่ต้องสลับ

เทียบ 45 กับ 55 >> ถูกแล้วไม่ต้องสลับ

ข้อมูลที่จะเรียงรอบที่ 2

9	18	12	32	25	45	50	55
---	----	----	----	----	----	----	----

ตัวอย่างการทำงาน

ข้อมูลที่จัดเรียงจากรอบที่ 2

9	18	12	32	25	45	50	55
---	----	----	----	----	----	----	----

รอบที่ 3 $\text{Gap} = 2/2 = 1$ ทำการเทียบแบบ Insertion Sort

เทียบ 9 กับ 18 >> ถูกแล้วไม่ต้องสลับ

เทียบ 18 กับ 12 >> สลับ 12 กับ 18

9	12	18	32	25	45	50	55
---	----	----	----	----	----	----	----

แล้วนำ 12 เทียบกับ 9 ถูกแล้วไม่ต้องสลับ

เทียบ 18 กับ 32 >> ถูกแล้วไม่ต้องสลับ

เทียบ 32 กับ 25 >> สลับ 25 กับ 32

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

แล้วนำ 25 เทียบกับ 18 ถูกแล้วไม่ต้องสลับ

ตัวอย่างการทำงาน

เทียบ 32 กับ 45 >> ถูกแล้วไม่ต้องสลับ

เทียบ 45 กับ 50 >> ถูกแล้วไม่ต้องสลับ

เทียบ 50 กับ 55 >> ถูกแล้วไม่ต้องสลับ

ข้อมูลที่จะเรียงเรียบร้อยแล้ว

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

ข้อแตกต่างระหว่าง Insertion Sort กับ Shell Sort

- **Insertion Sort:** ขยับเปรียบเทียบและขยับช่องว่างทีละ 1 ตำแหน่ง ทำให้ถ้าข้อมูลตัวน้อยๆ อยู่ขวาสุด ต้องเสียเวลาขยับถอยหลังหลายรอบมาก
- **Shell Sort:** นำ Insertion Sort มาปรับปรุงโดยการเปรียบเทียบและสลับข้ามช่องด้วยระยะทางกว้างๆ (Gap) ก่อน เพื่อดึงข้อมูลค่าน้อยมาฝั่งซ้ายอย่างรวดเร็ว แล้วค่อยทำ Insertion Sort ปิดท้าย จึงทำงานได้เร็วกว่ามากบนชุดข้อมูลขนาดใหญ่



5. การเรียงลำดับแบบผสาน (Merge Sort)

หลักการทำงาน

- Merge Sort เป็นอัลกอริทึมการเรียงลำดับที่ใช้กลยุทธ์ Divide and Conquer (การแบ่งแยกและเอาชนะ) โดยมีหลักการทำงานคือ "แบ่งชุดข้อมูลออกเป็นครึ่งหนึ่งย่อยๆ ลงไปเรื่อยๆ จนเหลือสมาชิกเพียงตัวเดียว แล้วจึงทำการนำข้อมูลย่อยเหล่านั้นกลับมารวมกัน (Merge) พร้อมกับจัดเรียงลำดับไปในตัว"
- Time Complexity : $O(n \log n)$ เสมอทุกกรณี ไม่ว่าข้อมูลจะเรียงมาแล้วหรือไม่ก็ตาม
- จุดเด่น มีประสิทธิภาพคงที่ เสถียรสูง เหมาะกับชุดข้อมูลขนาดใหญ่มาก หรือการจัดการข้อมูลใน External Memory / Linked List
- ข้อเสีย ต้องใช้หน่วยความจำชั่วคราวในการสร้างอาร์เรย์ย่อยสำหรับผสานข้อมูล

ขั้นตอนการทำงาน

- การเรียงลำดับจาก น้อยไปมาก ประกอบด้วย 3 ขั้นตอนหลัก:
 1. **Divide (แบ่งออก):** หาจุดกลางของอาร์เรย์ แล้วแบ่งชุดข้อมูลออกเป็น 2 ฝั่ง (ซ้ายและขวา) ทำการแบ่งย่อยแบบนี้ซ้ำๆ (Recursion) จนกว่าแต่ละกลุ่มย่อยจะเหลือสมาชิกเพียง 1 ตัว (ซึ่งถือว่าเรียงลำดับในตัวเองเรียบร้อยแล้ว)
 2. **Conquer (แก้ปัญหา):** ในกรณีที่กลุ่มย่อยเหลือสมาชิก 1 ตัว ให้เริ่มกระบวนการจัดเรียง
 3. **Combine / Merge (ผสานรวม):** นำกลุ่มย่อยที่เรียงแล้ว 2 กลุ่ม มาเปรียบเทียบทีละตัวแล้วนำมารวมกันเป็นกลุ่มใหม่ที่ใหญ่ขึ้น และเรียงลำดับถูกต้อง ทำซ้ำจนกระทั่งได้ข้อมูลชุดใหญ่ที่เรียงสมบูรณ์กลับมา

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

ขั้นที่ 1: การแบ่งย่อย (Divide)

- แบ่งครึ่งครั้งที่ 1:

25	32	12	55
----	----	----	----

9	18	50	45
---	----	----	----

- แบ่งครึ่งครั้งที่ 2:

25	32	12	55
----	----	----	----

9	18
---	----

50	45
----	----

- แบ่งจนเหลือ 1 ตัว:

25

32

12

55

9

18

50

45

ตัวอย่างการทำงาน

ขั้นที่ 2: การผสานและจัดเรียง (Merge)

25

32

12

55

9

18

50

45

การผสานจะนำสองฝั่งมาเปรียบเทียบตัวหน้าสุดแล้วดึงตัวที่น้อยกว่าลงมาวางเรียง

- รวมคู่ย่อย 1: ดึงข้อมูลมา 2 ตัว ทำการเปรียบเทียบกัน ค่าใดน้อยกว่าจะทำการใส่ค่านั้นลงไปก่อน

25	32
----	----

12	55
----	----

9	18
---	----

45	50
----	----

ตัวอย่างการทำงาน

- รวมคู่ย่อย 2: นำข้อมูล 2 ชุดมาทำการเปรียบเทียบข้อมูลตำแหน่ง i กับ j ค่าใดน้อยกว่าให้นำค่านั้นใส่ลงอาร์เรย์ และทำการเพิ่มค่า index 1 ค่า

25	32
----	----

i	
---	--

i	
---	--

	i
--	---

12	55
----	----

j	
---	--

	j
--	---

	j
--	---

--	--	--	--

12			
----	--	--	--

12	25		
----	----	--	--

12	25	32	
----	----	----	--

12	25	32	55
----	----	----	----

เมื่อ i ถึงตำแหน่งสุดท้ายแล้ว ก็จะนำข้อมูลในอีกชุดที่เหลือใส่ลงไปในอาร์เรย์

ตัวอย่างการทำงาน

- รวมคู่ย่อย 2: นำข้อมูล 2 ชุดมาทำการเปรียบเทียบข้อมูลตำแหน่ง i กับ j ค่าใดน้อยกว่าให้นำค่านั้นใส่ลงอาร์เรย์ และทำการเพิ่มค่า index 1 ค่า

9	18
---	----

45	50
----	----

--	--	--	--

i	
-----	--

j	
-----	--

9			
---	--	--	--

	i
--	-----

j	
-----	--

9	18		
---	----	--	--

- เมื่อ i ถึงตำแหน่งสุดท้ายแล้ว ก็จะนำข้อมูลในอีกชุดที่เหลือใส่ลงไปในอาร์เรย์

9	18	45	50
---	----	----	----

ตัวอย่างการทำงาน

- รวมขั้นตอนสุดท้าย:

12	25	32	55
i			
i			
	i		
	i		

9	18	45	50
j			
	j		
	j		
		j	

9							
9	12						
9	12	18					
9	12	18	25				

ตัวอย่างการทำงาน

- รวมขั้นตอนสุดท้าย:

12	25	32	55
----	----	----	----

		i	
--	--	---	--

			i
--	--	--	---

			i
--	--	--	---

9	18	45	50
---	----	----	----

		j	
--	--	---	--

--	--	--	--

			j
--	--	--	---

9	12	18	25	32			
---	----	----	----	----	--	--	--

9	12	18	25	32	45		
---	----	----	----	----	----	--	--

9	12	18	25	32	45	50	
---	----	----	----	----	----	----	--

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----



6. การเรียงลำดับแบบเร็ว (Quick Sort)

หลักการทำงาน

- Quick Sort (การเรียงลำดับแบบเร็ว) เป็นอัลกอริทึมการเรียงลำดับแบบ Divide and Conquer (แบ่งแยกและเอาชนะ) ที่ได้รับความนิยมสูงมาก โดยมีหลักการทำงานสำคัญคือ "เลือกสมาชิกขึ้นมา 1 ตัวเพื่อเป็นจุดหมุน (Pivot) แล้วจัดกลุ่มข้อมูลใหม่ให้ตัวที่น้อยกว่า Pivot ไปอยู่ทางซ้าย และตัวที่มากกว่าไปอยู่ทางขวา" จากนั้นทำซ้ำกระบวนการนี้กับกลุ่มข้อมูลทางซ้ายและขวาเรื่อย ๆ (Recursion)
- Best / Average Time Complexity : $O(n \log n)$ ทำงานได้เร็วมากในสถานะทั่วไป
- Worst Time Complexity : $O(n^2)$ เกิดขึ้นเมื่อเลือก Pivot ได้แย่ที่สุด เช่น ข้อมูลเรียงมาอยู่แล้วแต่เลือกตัวแรก/ตัวสุดท้ายเป็น Pivot เสมอ

ขั้นตอนการทำงานการเรียงลำดับจาก น้อยไปมาก

1. เลือก **Pivot**: เลือกข้อมูลมา 1 ตัวจากอาร์เรย์ (นิยมเลือกตัวแรก, ตัวสุดท้าย หรือ ตัวตรงกลาง)

2. จัดกลุ่ม (**Partitioning**): เปรียบเทียบเพื่อจัดกลุ่มข้อมูลออกเป็น 2 กลุ่ม คือ

- ข้อมูลทุกตัวที่มีค่าน้อยกว่า Pivot จะอยู่ฝั่งซ้าย
- ข้อมูลทุกตัวที่มีค่ามากกว่า Pivot จะอยู่ฝั่งขวา

แล้วนำ Pivot ไปวางไว้ตรงกลางระหว่างสองฝั่ง (ณ ตำแหน่งนี้ Pivot จะถือว่าอยู่ในตำแหน่งที่ถูกต้องสมบูรณ์แล้ว)

ขั้นตอนการทำงานการเรียงลำดับจาก น้อยไปมาก

ขั้นตอนการจัดกลุ่ม

2.1 ทำการกำหนดตัวแปร i เป็นตำแหน่งแรกของชุดข้อมูล และ j เป็นตำแหน่งสุดท้ายของชุดข้อมูล

2.2. ทำการเปรียบเทียบข้อมูลตำแหน่งที่ i กับ pivot โดย

- หากค่าตำแหน่ง $i < \text{pivot}$ ให้ทำการเพิ่มค่า i ไป 1
- หากค่าตำแหน่ง $i > \text{pivot}$ ให้หยุด แล้วไปพิจารณาที่ตำแหน่ง j

2.3 ทำการเปรียบเทียบข้อมูลตำแหน่งที่ j กับ pivot โดย

- หากค่าตำแหน่ง $j > \text{pivot}$ ให้ทำการลดค่า j ลง 1
- หากค่าตำแหน่ง $j < \text{pivot}$ ให้หยุด

ขั้นตอนการทำงานการเรียงลำดับจาก น้อยไปมาก

2.4 เมื่อ i และ j หยุดแล้ว

- ถ้า $i < j$ ให้ทำการสลับข้อมูลตำแหน่ง i และ j แล้วเพิ่มค่า i ไป 1 แล้วทำซ้ำขั้นตอนที่ 2.2-2.4
- ถ้า $i > j$ ให้หยุด แล้วทำการสลับค่า pivot กับค่าที่ตำแหน่ง i และจบรอบการทำงาน

3. ทำซ้ำแบบ **Recursion**: นำกลุ่มข้อมูลฝั่งซ้ายและฝั่งขวาที่ยังไม่ได้เรียงลำดับ มาทำขั้นตอนที่ 1-2 ซ้ำไปเรื่อยๆ จนกว่ากลุ่มย่อยจะเหลือสมาชิกเพียง 1 ตัวหรือไม่มีเลย

ตัวอย่างการทำงาน

- ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

รอบที่ 1

ขั้นตอนที่ 1 ทำการเลือกค่า pivot ในที่นี้จะเลือกจากข้อมูลตัวสุดท้าย pivot = 45

ขั้นตอนที่ 2 การจัดเรียง

กำหนด i เป็นตัวแรกและ j เป็นตัวสุดท้ายของชุดข้อมูล

1. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $25 < 45$ เลื่อน i
2. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $32 < 45$ เลื่อน i
3. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $12 < 45$ เลื่อน i
4. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $55 > 45$ หยุด ไปพิจารณา j

25	32	12	55	9	18	50	45
i						j	
	i						
		i					
			i			j	

ตัวอย่างการทำงาน

ขั้นตอนที่ 2 การจัดเรียง

5. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $50 > 45$ เลื่อน j ลง
6. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $18 < 45$ หยุด
สลับค่าตำแหน่ง i และ j และเพิ่มค่า i
7. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $9 < 45$ เลื่อน i
8. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $55 > 45$ หยุด ไปพิจารณา j
9. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $55 > 45$ เลื่อน j ลง
10. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $9 < 45$ หยุด
ตอนนี้ $j < i$ ให้ทำการสลับ pivot กับ i (จบรอบที่ 1)

25	32	12	55	9	18	50	45
			i			j	
			i		j		
25	32	12	18	9	55	50	45
				i	j		
					ij		
				j	i		
25	32	12	18	9	45	50	55

ตัวอย่างการทำงาน

รอบที่ 2 ข้อมูลฝั่งซ้ายของ 45

ขั้นตอนที่ 1 เลือก pivot = 9

ขั้นตอนที่ 2 การจัดเรียง

กำหนด i เป็นตัวแรกและ j เป็นตัวสุดท้ายของชุดข้อมูล

1. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $25 > 9$ หยุด ไปพิจารณา j
 2. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $18 > 9$ เลื่อน j ลง
 3. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $12 > 9$ เลื่อน j ลง
 4. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $32 > 9$ เลื่อน j ลง
 5. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $25 > 9$ เลื่อน j ลง
- ตอนนี้ $j < i$ ให้ทำการสลับ pivot กับ i (จบรอบที่ 2)

25	32	12	18	9	45	50	55
----	----	----	----	---	----	----	----

25	32	12	18	9
i			j	
i		j		
i	j			
i j				
9	25	32	12	18

ตัวอย่างการทำงาน

รอบที่ 2 ข้อมูลฝั่งซ้ายของ 45

ขั้นตอนที่ 1 เลือก pivot = 9

ขั้นตอนที่ 2 การจัดเรียง

กำหนด i เป็นตัวแรกและ j เป็นตัวสุดท้ายของชุดข้อมูล

1. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $25 > 9$ หยุด ไปพิจารณา j
 2. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $18 > 9$ เลื่อน j ลง
 3. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $12 > 9$ เลื่อน j ลง
 4. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $32 > 9$ เลื่อน j ลง
 5. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $25 > 9$ เลื่อน j ลง
- ตอนนี้ $j < i$ ให้ทำการสลับ pivot กับ i (จบรอบที่ 2)

9	25	32	12	18	45	50	55
---	----	----	----	----	----	----	----

25	32	12	18	9
i			j	
i		j		
i	j			
i j				
9	25	32	12	18

ตัวอย่างการทำงาน

รอบที่ 3 ข้อมูลฝั่งขวาของ 9

ขั้นตอนที่ 1 เลือก pivot = 18

ขั้นตอนที่ 2 การจัดเรียง

กำหนด i เป็นตัวแรกและ j เป็นตัวสุดท้ายของชุดข้อมูล

1. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $25 > 18$ หยุด ไปพิจารณา j

2. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $12 < 18$ หยุด

สลับค่าตำแหน่ง i และ j และเพิ่มค่า i

3. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $32 > 18$ หยุด ไปพิจารณา j

9	25	32	12	18	45	50	55
---	----	----	----	----	----	----	----

25	32	12	18
i		j	
i		j	
12	32	25	18
	i	j	

ตัวอย่างการทำงาน

ขั้นตอนที่ 2 การจัดเรียง (ต่อ)

4. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $25 > 18$ เลื่อน j ลง

5. เปรียบเทียบค่าตำแหน่ง j กับ pivot : $32 > 18$ เลื่อน j ลง

ตอนนี้ $j < i$ ให้ทำการสลับ pivot กับ i (จบรอบที่ 3)

9	25	32	12	18	45	50	55
---	----	----	----	----	----	----	----

12	32	25	18
	i	j	
	ij		
j	i		
12	18	25	32

ตัวอย่างการทำงาน

รอบที่ 4 ข้อมูลฝั่งขวาของ 18

ขั้นตอนที่ 1 เลือก pivot = 32

ขั้นตอนที่ 2 การจัดเรียง

กำหนด i เป็นตัวแรกและ j เป็นตัวสุดท้ายของชุดข้อมูล

1. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $25 < 32$ เลื่อน i

ตอนนี้ $j < i$ ให้ทำการสลับ pivot กับ i (จบรอบที่ 4)

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

25	32
i j	
j	i
25	32

ตัวอย่างการทำงาน

รอบที่ 5 ข้อมูลฝั่งขวาของ 45

ขั้นตอนที่ 1 เลือก pivot = 55

ขั้นตอนที่ 2 การจัดเรียง

กำหนด i เป็นตัวแรกและ j เป็นตัวสุดท้ายของชุดข้อมูล

1. เปรียบเทียบค่าตำแหน่ง i กับ pivot : $50 < 55$ เลื่อน i

ตอนนี้ $j < i$ ให้ทำการสลับ pivot กับ i (จบรอบที่ 5)

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

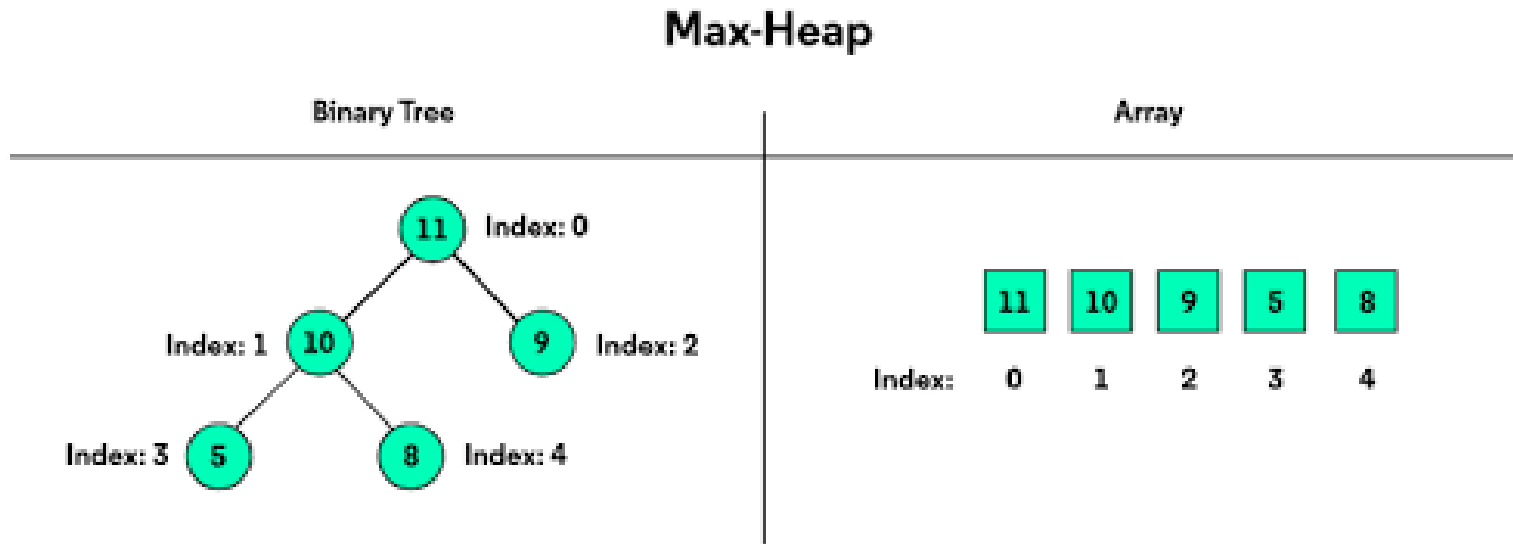
50	55
i j	
j	i
50	55

9	12	18	25	32	45	50	55
---	----	----	----	----	----	----	----

7. การเรียงลำดับแบบฮีป (Heap Sort)

หลักการทำงาน

- Heap Sort (การเรียงลำดับแบบฮีป) เป็นอัลกอริทึมการเรียงลำดับที่ประยุกต์ใช้โครงสร้างข้อมูลแบบ Binary Heap เข้ามาช่วย มีหลักการทำงานคือ "เปลี่ยนอาร์เรย์ให้อยู่ในรูป Max-Heap เพื่อให้ตัวที่มากที่สุดมาอยู่ที่ราก (Root) แล้วดึงตัวนั้นไปไว้ท้ายสุด จากนั้นจัดระเบียบฮีปใหม่ทำซ้ำไปเรื่อย ๆ"



หลักการทำงาน

- โครงสร้างข้อมูล Heap คืออะไร

ในการเรียงลำดับจากน้อยไปมาก เราจะใช้ Max-Heap ซึ่งมีคุณสมบัติสำคัญคือ:

1. เป็น Binary Tree ที่ โหนดพ่อแม่จะมีค่ามากกว่าหรือเท่ากับ โหนดลูกเสมอ
2. ข้อมูลที่มีค่าน้อยหรือมากที่สุดในกลุ่ม จะถูกดันมาอยู่ที่ Root (โหนดบนสุด / Index 0 ของ Array) เสมอ

- Time Complexity : $O(n \log n)$ เสมอทุกกรณี ไม่ขึ้นกับว่าข้อมูลจะเรียงมาแล้วหรือไม่

ขั้นตอนการทำงานการเรียงลำดับจาก น้อยไปมาก

1. การสร้าง Max-Heap (Build Max-Heap):

- จัดเรียงข้อมูลในอาร์เรย์เดิมให้มีโครงสร้างเป็น Max-Heap
- เมื่อสร้างเสร็จ ข้อมูลที่มีค่ามากที่สุดในการ์เรย์จะมาอยู่ที่ตำแหน่งแรกสุด (Index 0) ทันที

2. สลับค่านำตัวมากที่สุดไปไว้ท้าย (Swap):

- สลับข้อมูลตำแหน่งแรกสุด (Index 0 ซึ่งมากที่สุด) กับข้อมูลตำแหน่งสุดท้ายของ Heap
- ตัดตำแหน่งสุดท้ายนั้นออกจาก Heap (ถือว่าตัวนั้นเรียงลำดับเสร็จสมบูรณ์แล้ว)

ขั้นตอนการทำงานการเรียงลำดับจาก น้อยไปมาก

3. ปรับสมดุล Heap ใหม่ (Heapify):

- เนื่องจากตัวที่สลับขึ้นไปอยู่ตำแหน่งแรก (Index 0) อาจไม่ได้มีค่ามากที่สุด ทำให้คุณสมบัติ Max-Heap เสียไป จึงต้องจัดโครงสร้างให้กลับมาเป็น Max-Heap อีกครั้ง (กระบวนการ Heapify)
- ตัวที่มากที่สุดในกลุ่มที่เหลือจะกลับไปอยู่ที่ Index 0 อีกครั้ง

4. ทำซ้ำ:

- ทำตามขั้นตอนที่ 2 และ 3 ไปเรื่อย ๆ จนกว่าสมาชิกใน Heap จะเหลือเพียง 1 ตัว

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

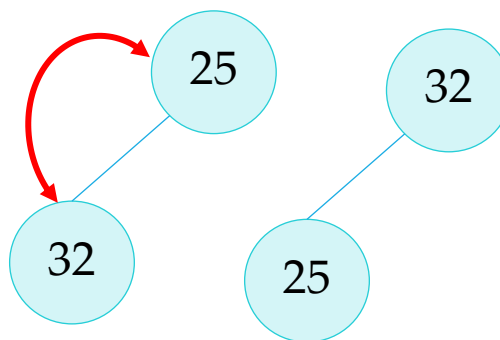
รอบที่ 1

ขั้นตอนที่ 1 สร้าง Max-Heap

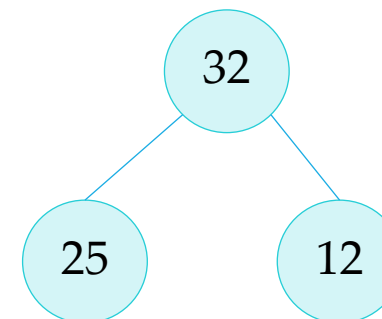
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



Input 25



Input 32

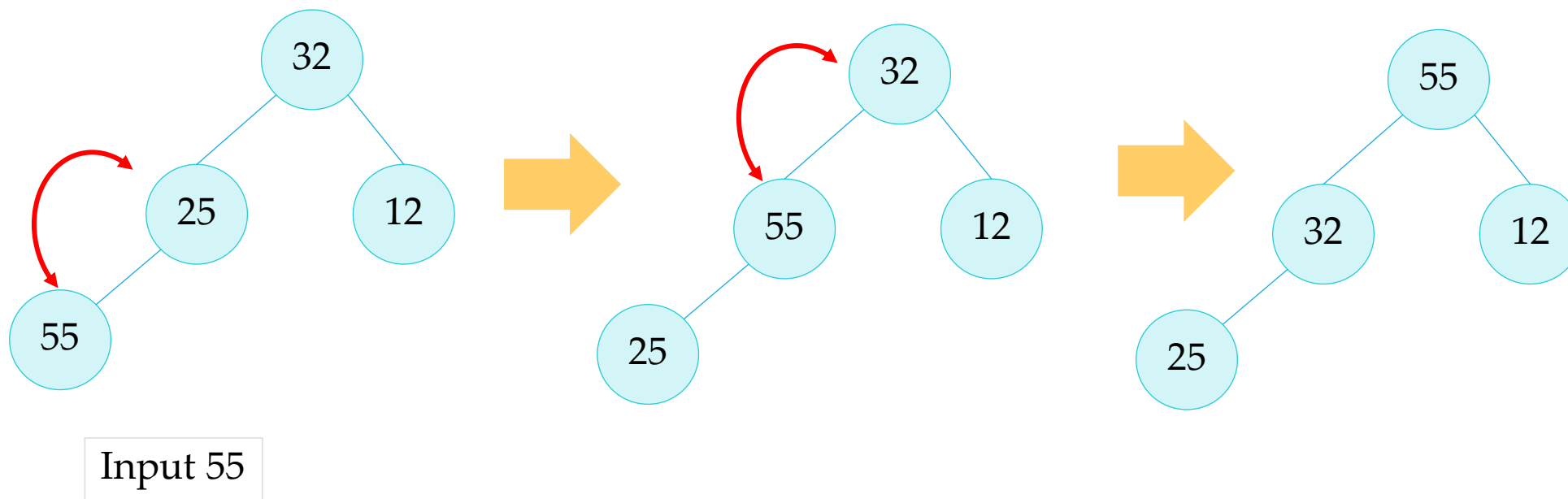


Input 12

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

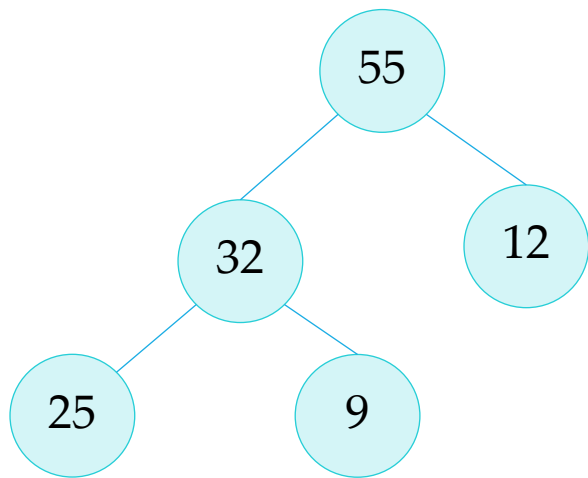
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



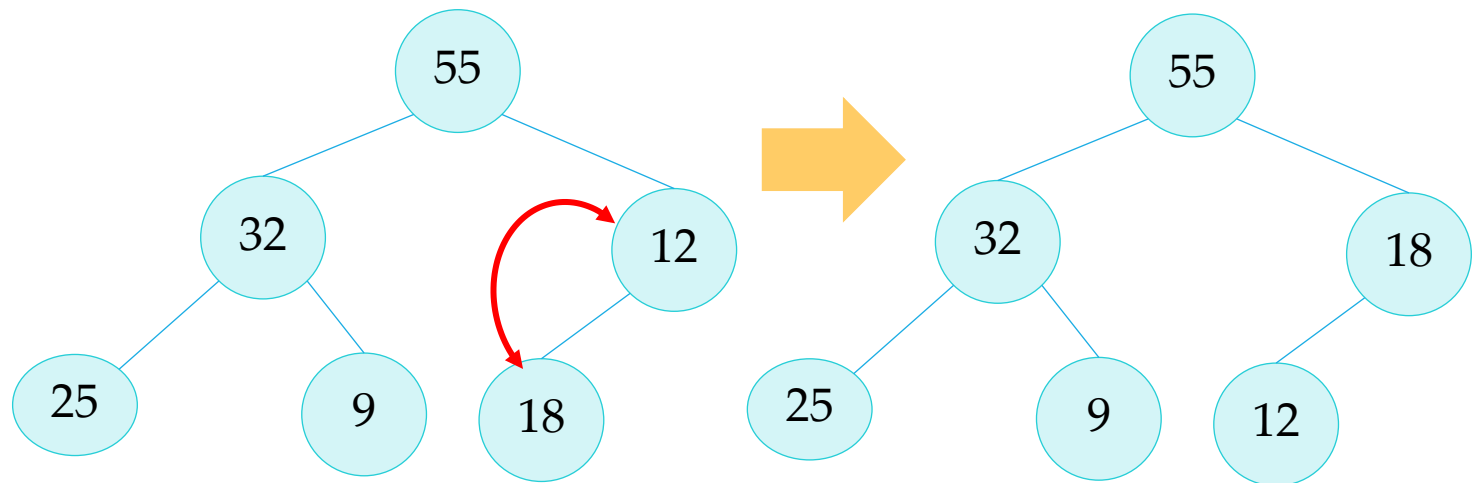
ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



Input 9

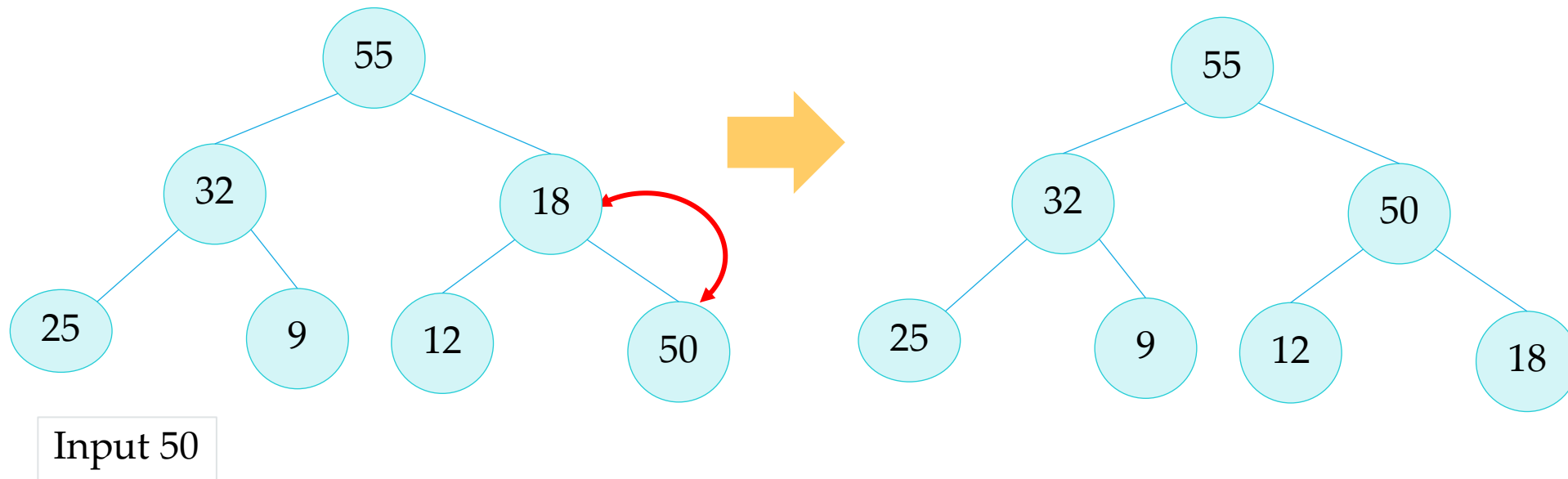


Input 18

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

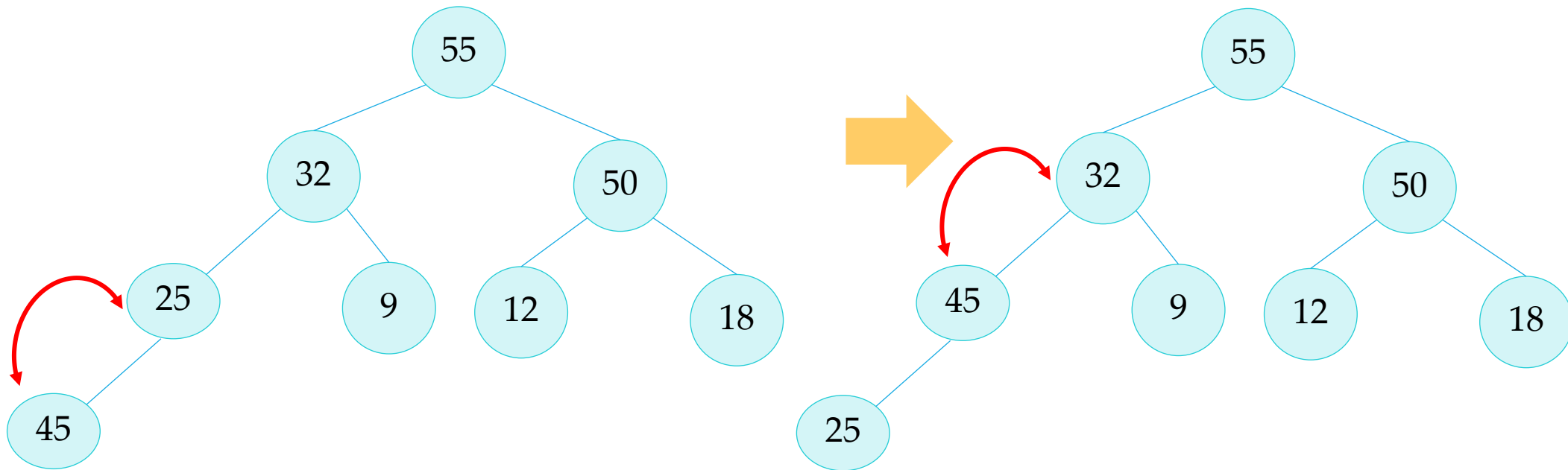
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

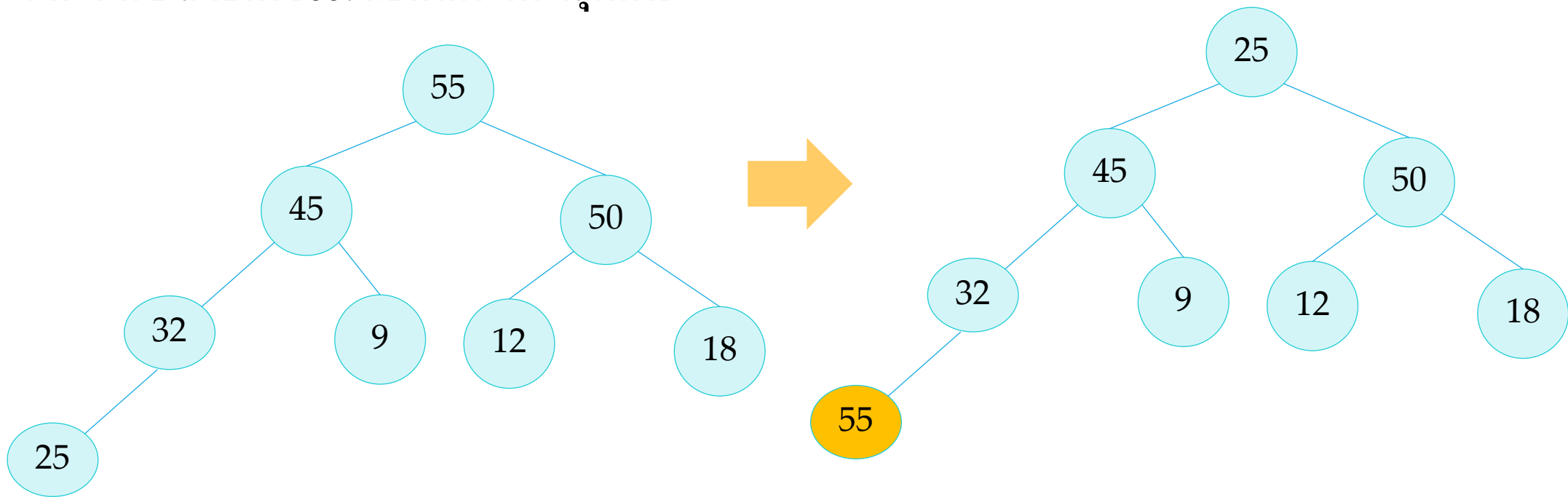


Input 45

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
ขั้นตอนที่ 2 สลับค่า root กับค่าตำแหน่งสุดท้าย

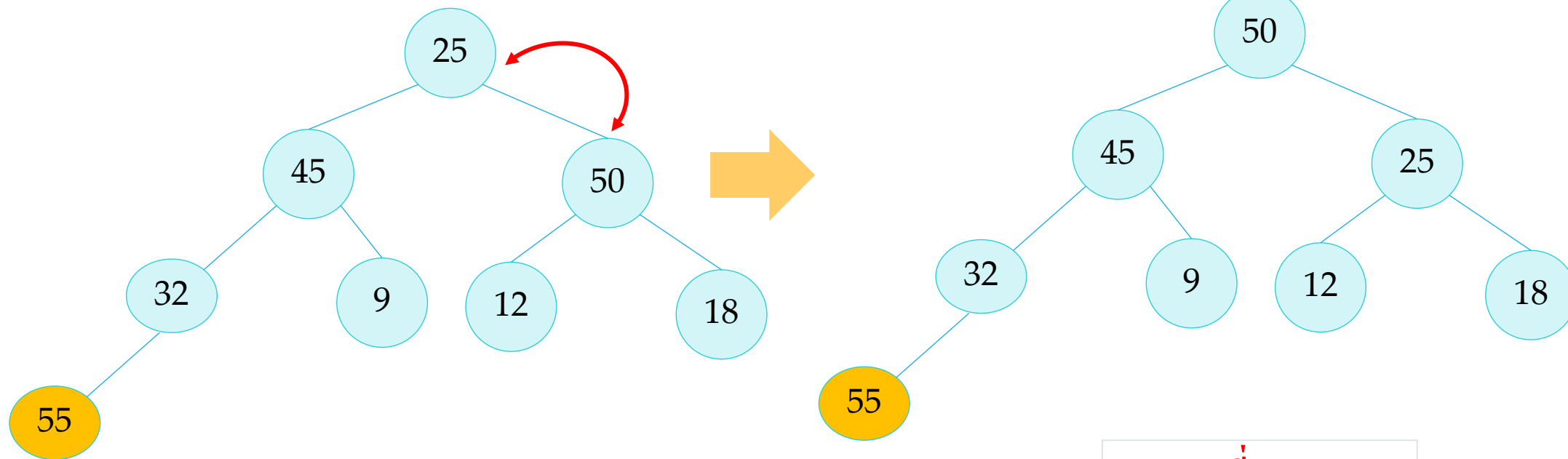
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

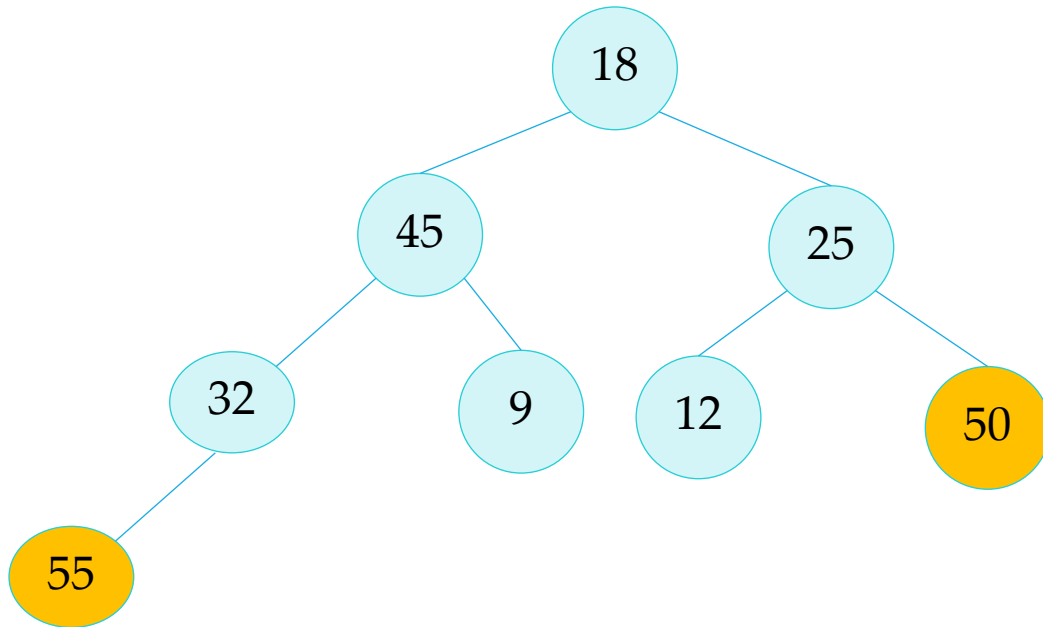


จบรอบที่ 1

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 2 ขั้นตอนที่ 2 สลับค่า root กับค่าตำแหน่งสุดท้าย

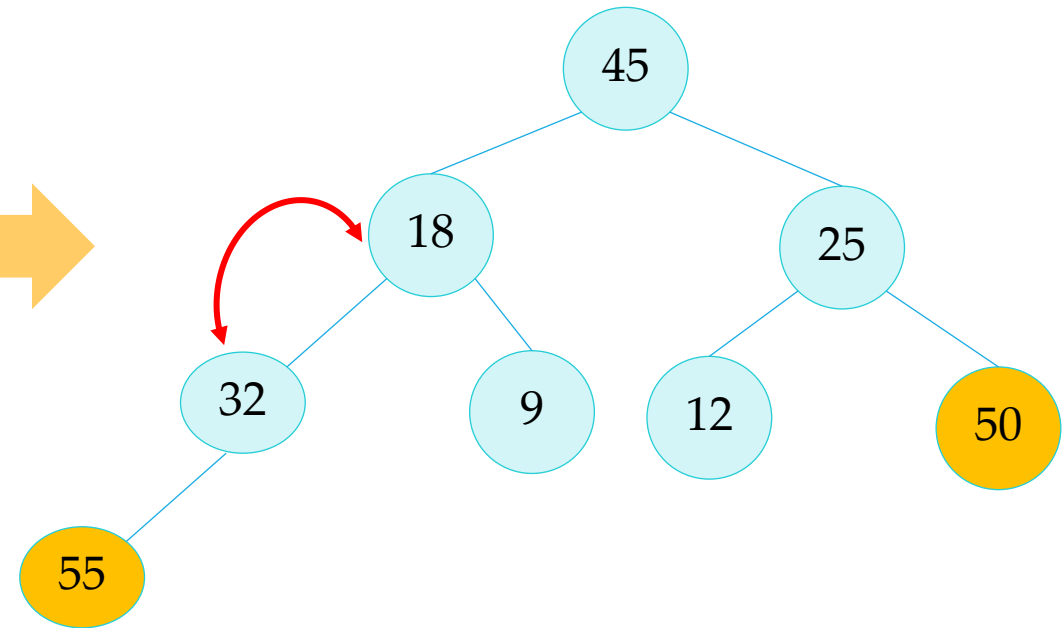
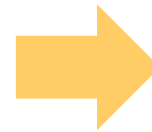
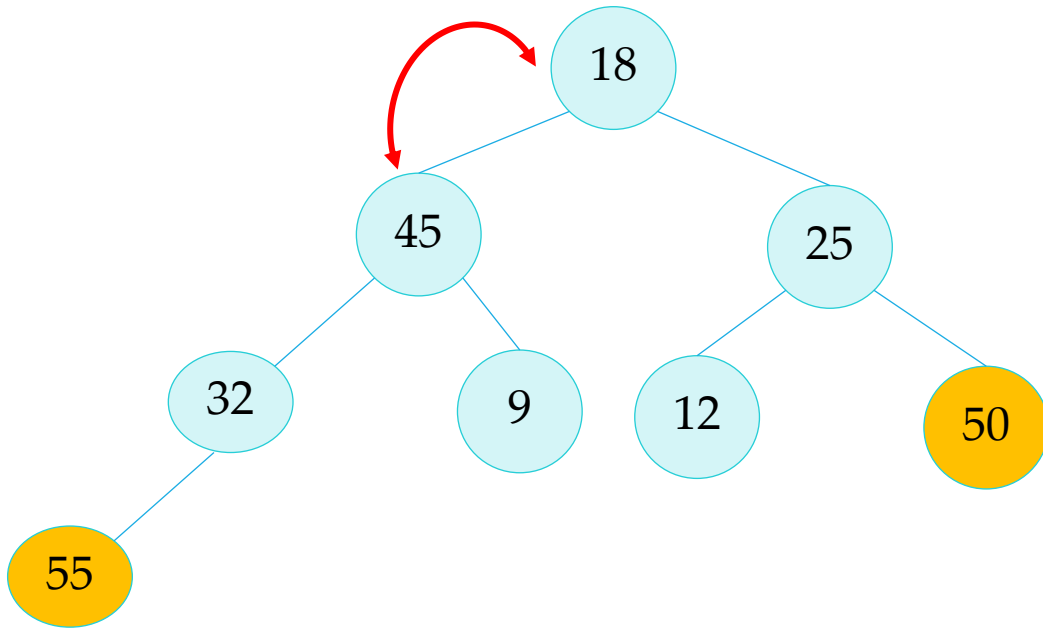
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 2 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

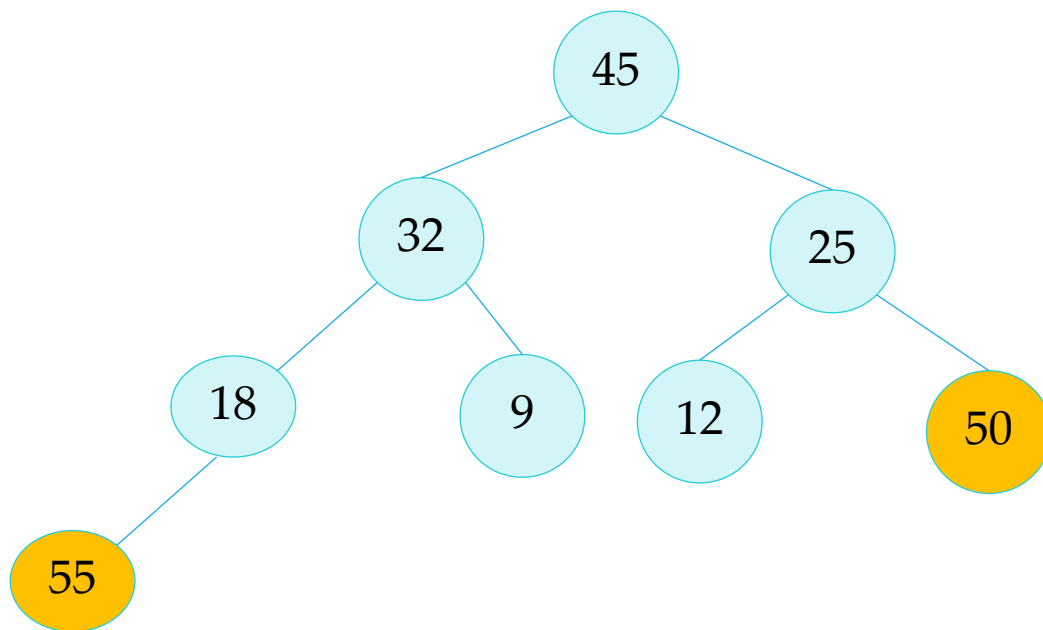
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 2 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

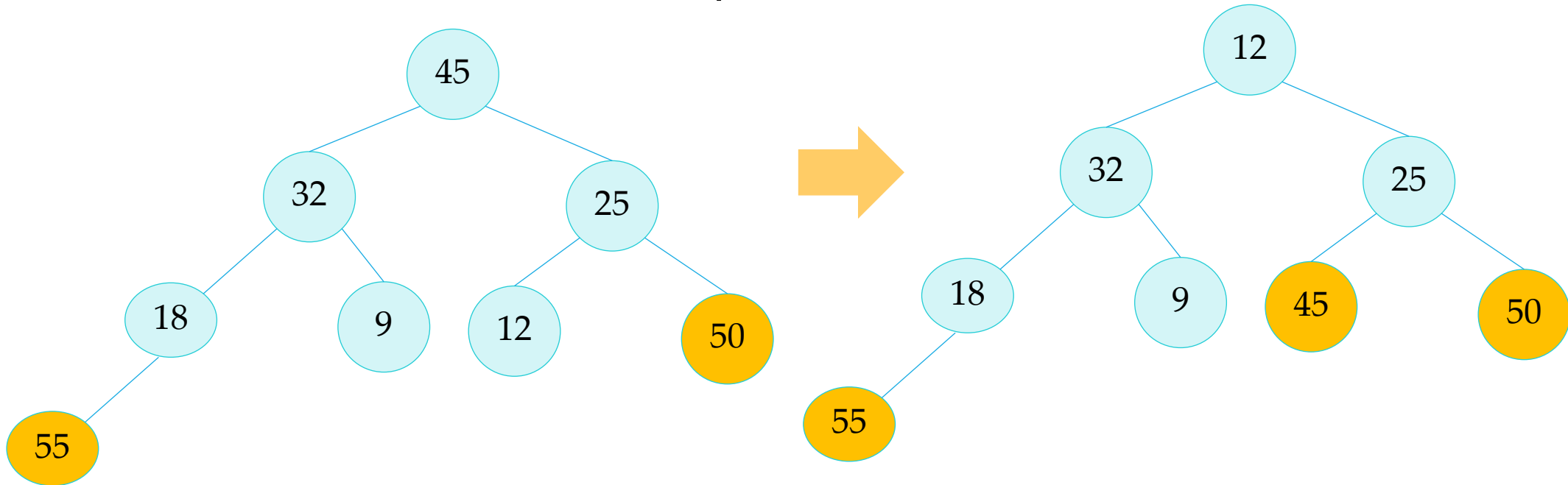


จบรอบที่ 2

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 3 ขั้นตอนที่ 2 สลับค่า root กับค่าตำแหน่งสุดท้าย

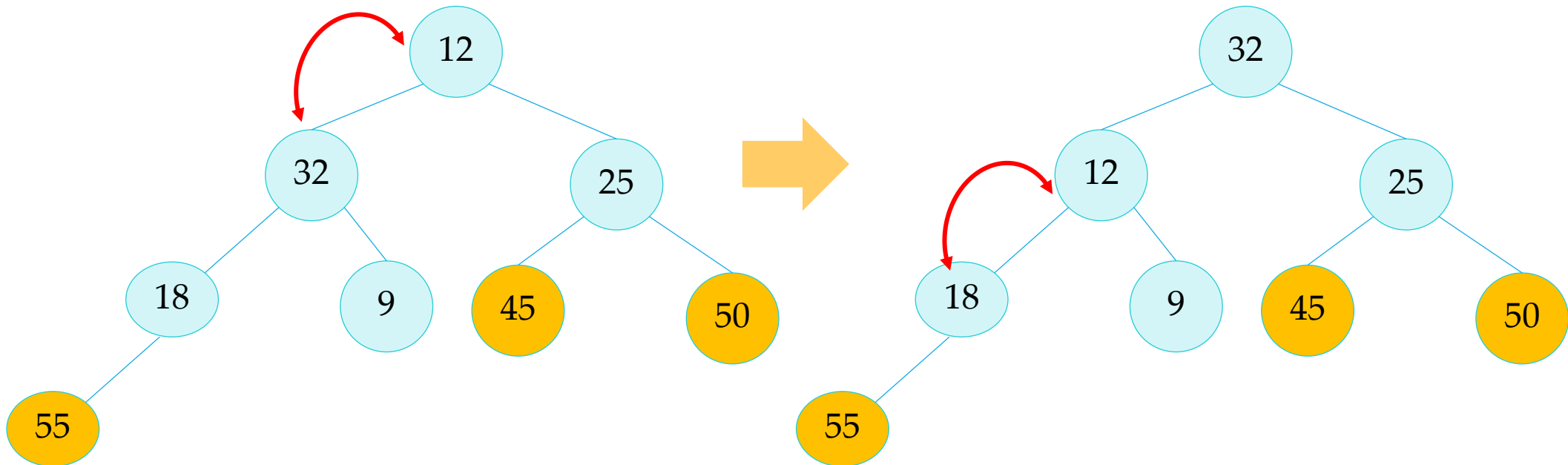
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 3 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

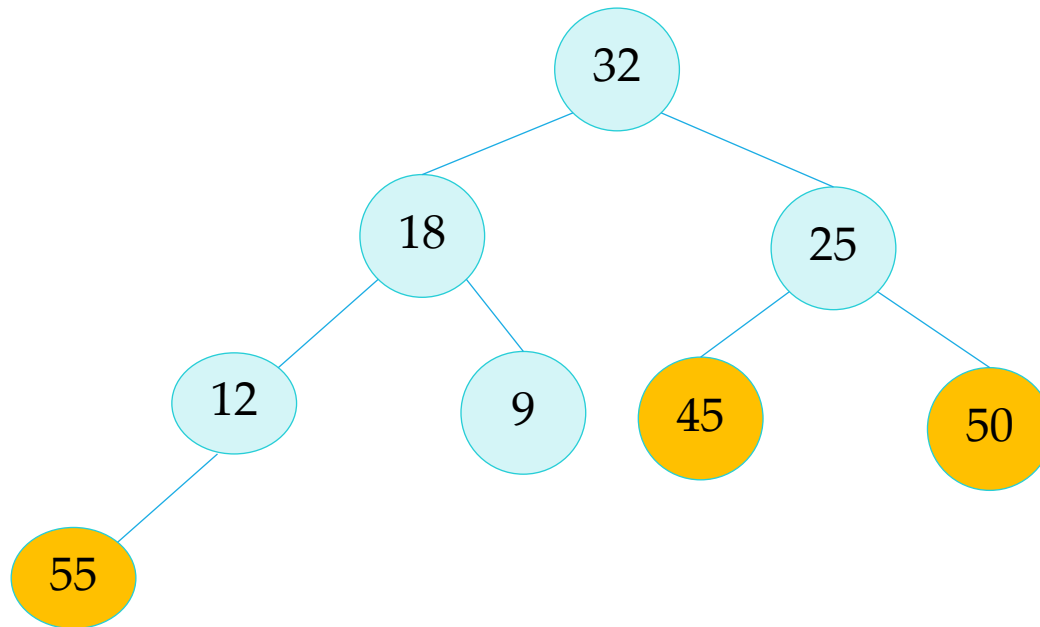
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 3 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

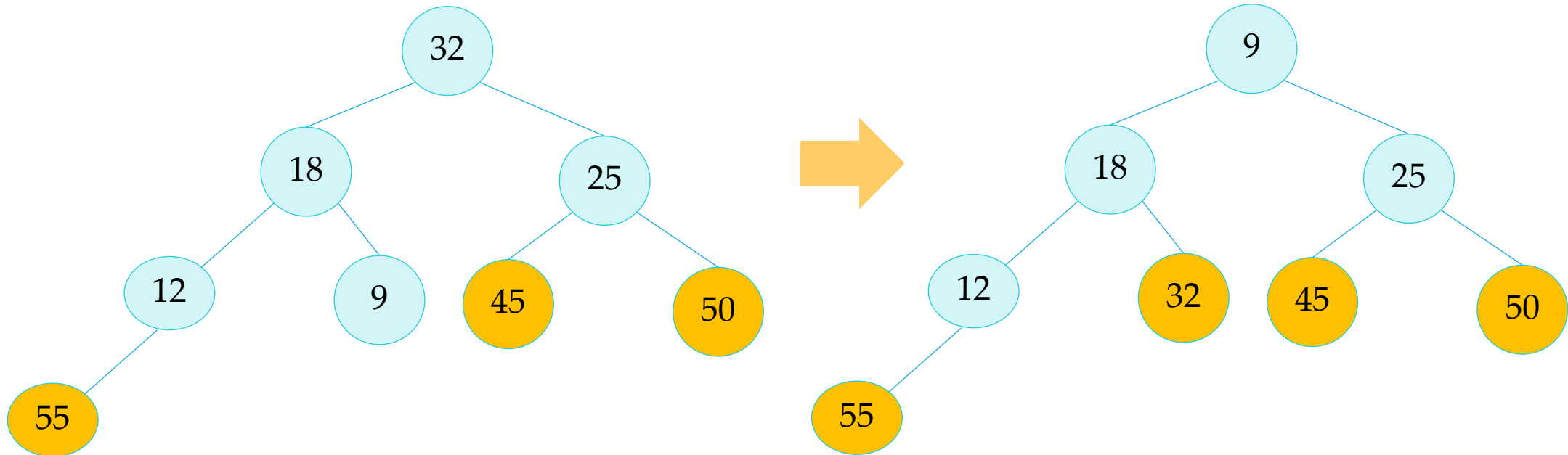


จบรอบที่ 3

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 4 ขั้นตอนที่ 2 สลับค่า root กับค่าตำแหน่งสุดท้าย

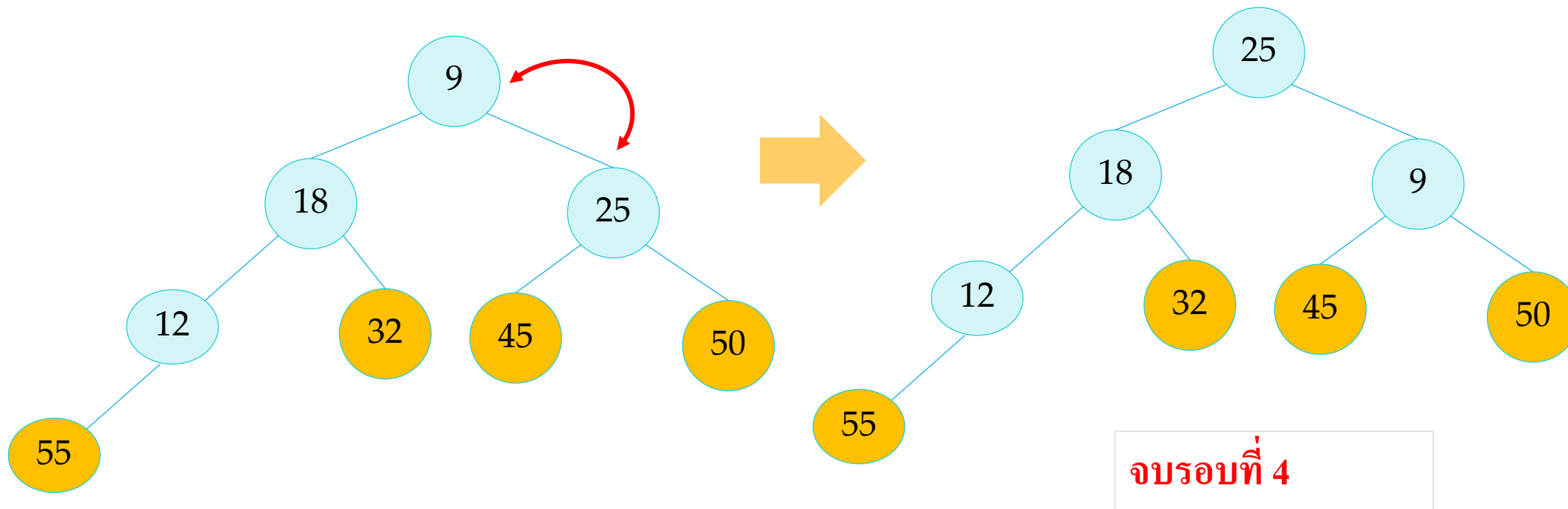
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 4 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

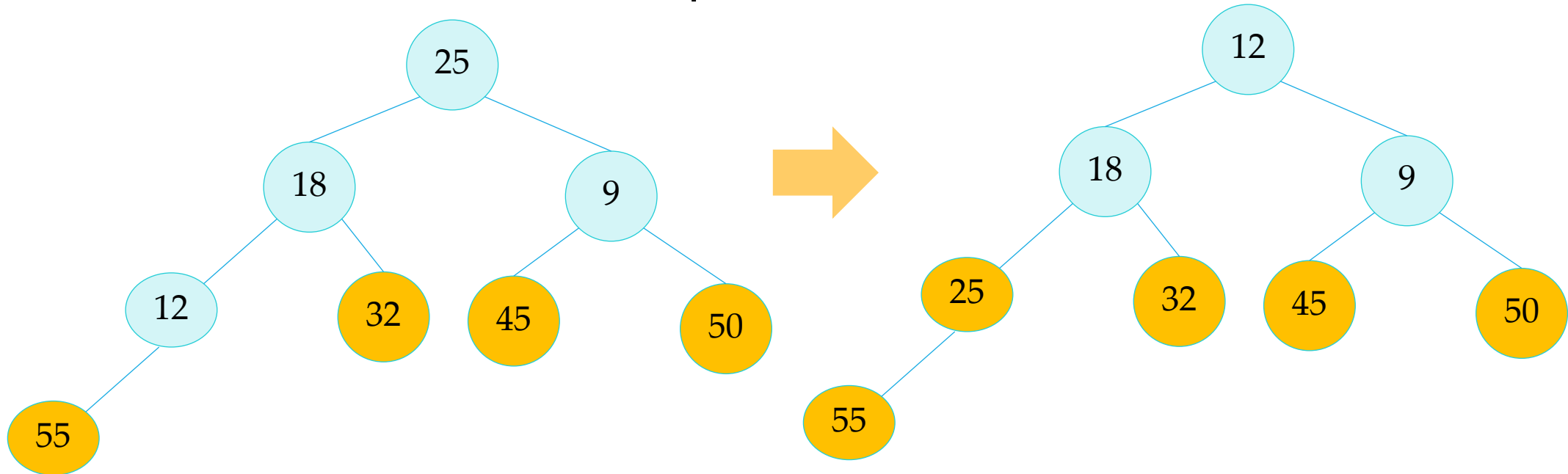
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 5 ขั้นตอนที่ 2 สลับค่า root กับค่าตำแหน่งสุดท้าย

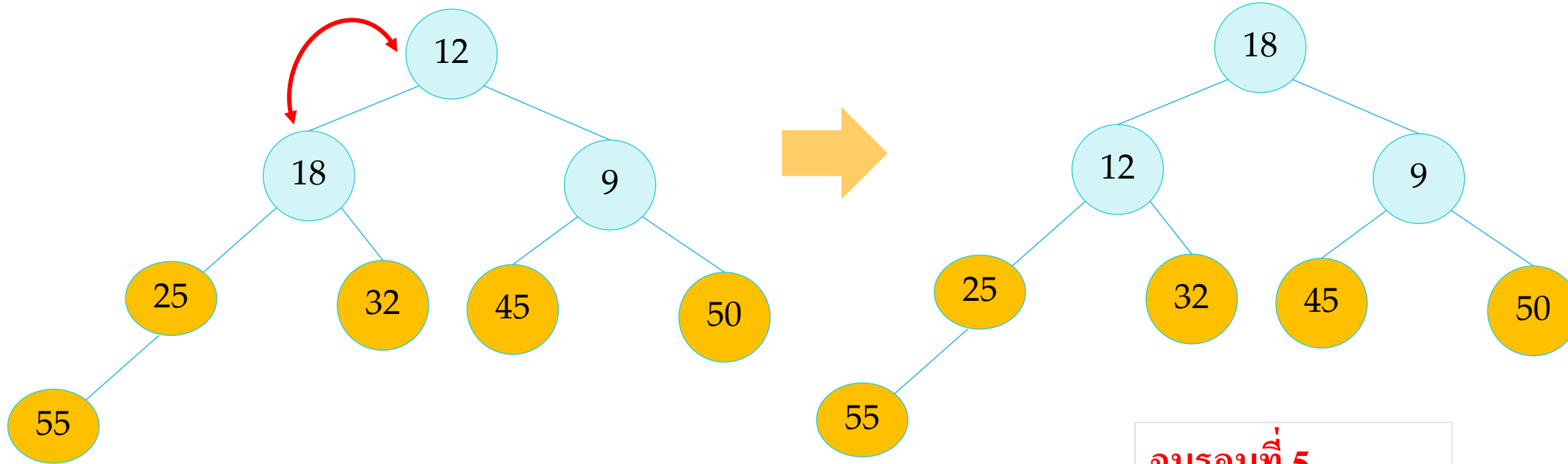
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 5 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

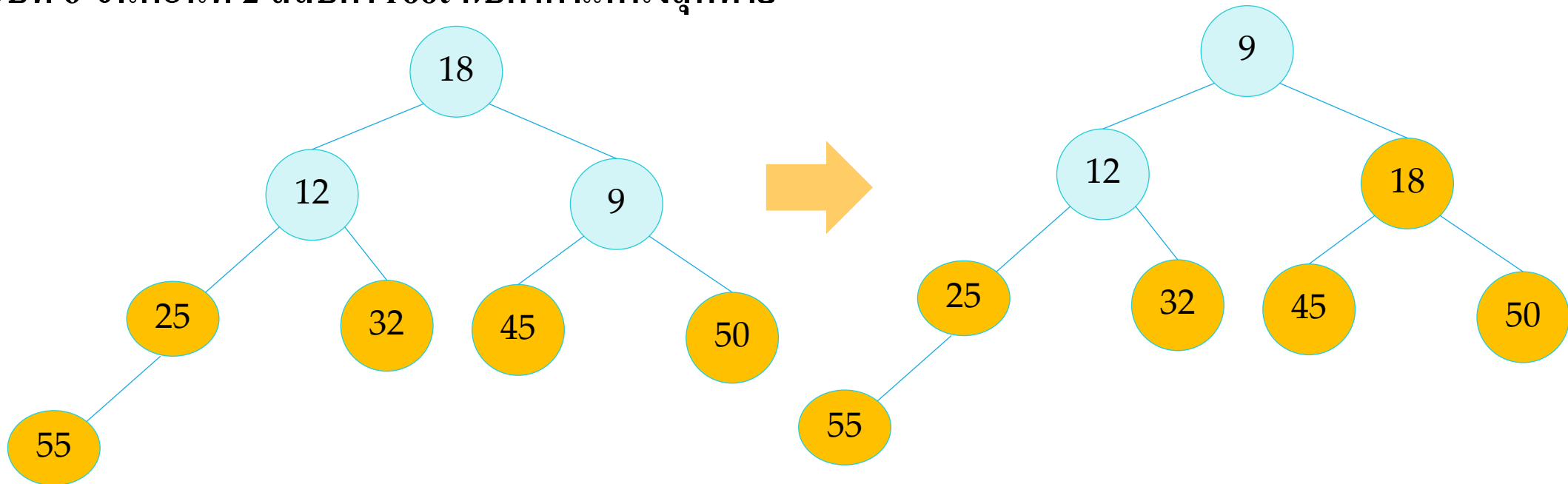


จบรอบที่ 5

ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 6 ขั้นตอนที่ 2 สลับค่า root กับค่าตำแหน่งสุดท้าย

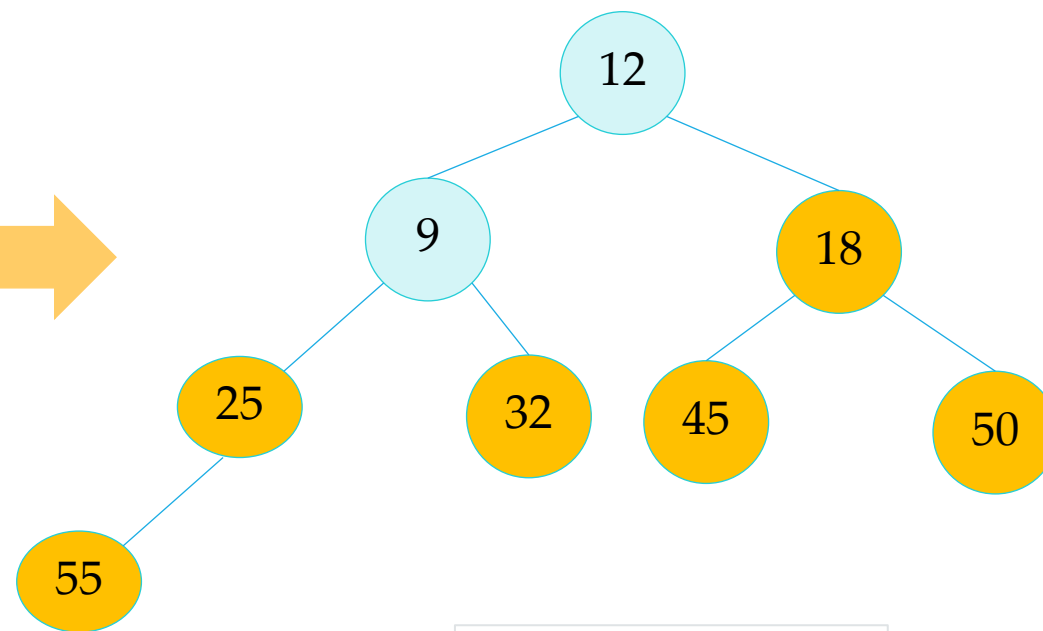
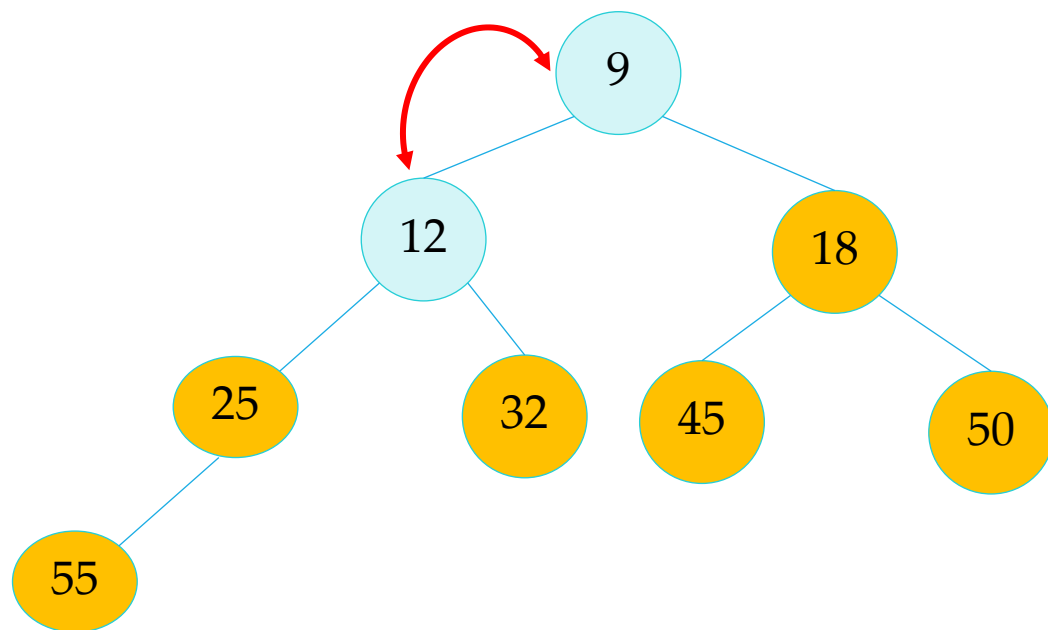
25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 6 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

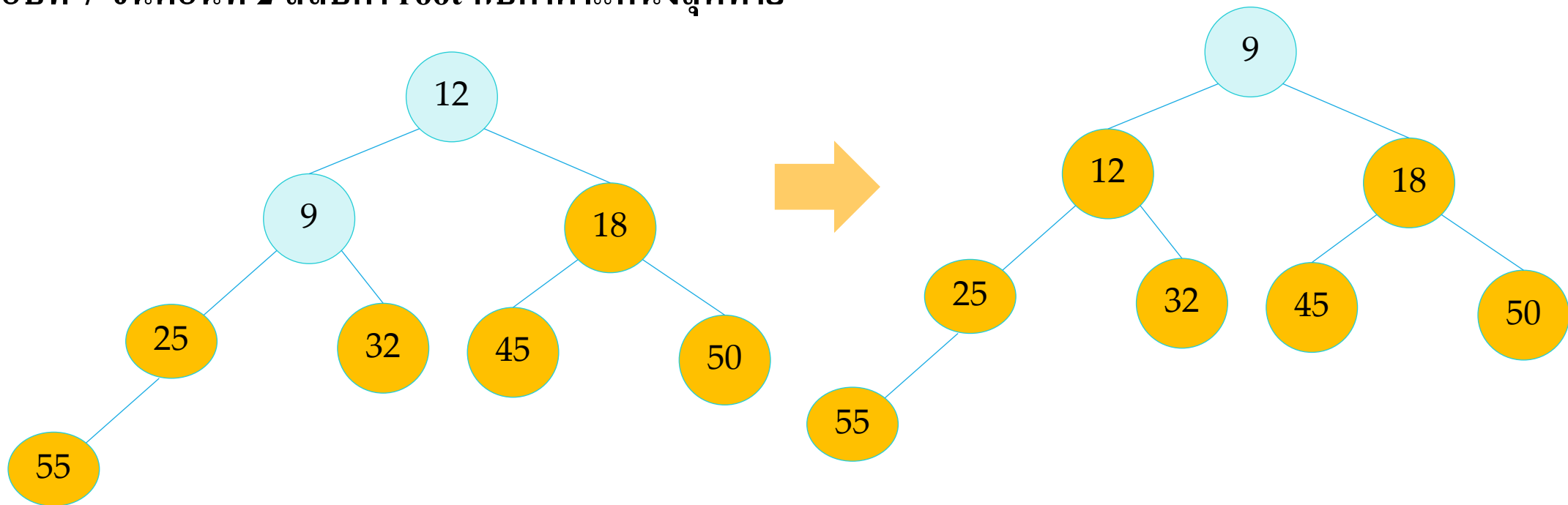


จบรอบที่ 6

ตัวอย่างการทำงาน

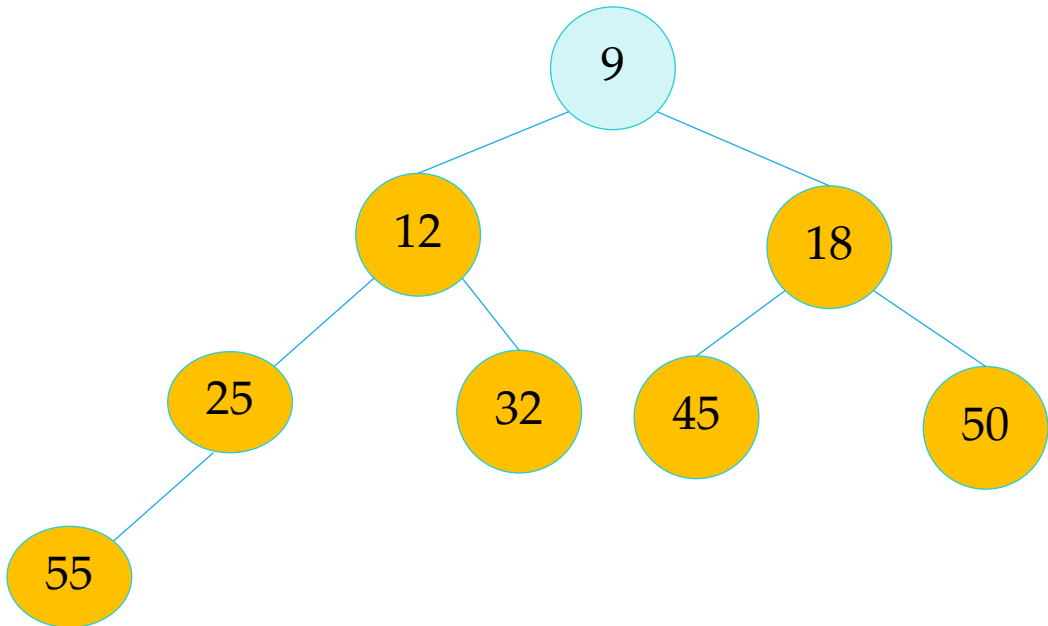
ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 7 ขั้นตอนที่ 2 สลับค่า root กับค่าตำแหน่งสุดท้าย

25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----



ตัวอย่างการทำงาน

ต้องการเรียงจากน้อยไปมาก สมมติมีข้อมูลชุดหนึ่งคือ
รอบที่ 7 ขั้นตอนที่ 3 ปรับโครงสร้าง Heap



25	32	12	55	9	18	50	45
----	----	----	----	---	----	----	----

เหลือสมาชิก 1 ตัว จบการทำงาน

ได้ข้อมูลที่จัดเรียงเรียบร้อยแล้ว

ข้อมูลที่จัดเรียงจะอ่านจาก root ไล่ลงไป แต่ละ level อ่านจากซ้ายไปขวา ตามหลักการ
ทำงานของ Complete binary tree

9 12 18 25 32 45 50 55